

以下は、「Hello, world!」と表示する簡単なプログラムです：

```
fun main() {  
    println("Hello, world!")  
    // Hello, world!  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="hello-world-kotlin"}
```

Kotlinでは：

- `fun` は関数を宣言するために使用されます
- `main()` 関数は、プログラムが開始される場所です
- 関数の本体は中括弧 `{}` で囲んで記述します。
- `println()` および `print()` 関数は、引数を標準出力に出力します。

関数とは、特定のタスクを実行するためのひと綴りの命令のことです。一度関数を作成しておけば、そのタスクを実行する必要があるときはいつでも、その命令をいちいち書き直すことなく利用できます。関数については、後の章でさらに詳しく説明します。それまでは、すべての例で `main()` 関数を使用します。

変数

すべてのプログラムにはデータを保存する機能が必要ですが、変数を使えばそれが可能になります。Kotlinでは、次のように宣言できます：

- `val` を使った読取り専用変数
- `var` を使った変更可能な変数

読み取り専用変数には、一度値を代入すると、その値を変更することはできません。

{style="note"}

値を代入するには、代入演算子 `=` を使用します。

例えば：

```
fun main() {  
    //sampleStart  
    val popcorn = 5    // There are 5 boxes of popcorn  
    val hotdog = 7     // There are 7 hotdogs  
    var customers = 10 // There are 10 customers in the queue  
  
    // Some customers leave the queue  
    customers = 8  
    println(customers)  
    // 8  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-variables"}
```

変数は、プログラムの冒頭で `main()` 関数の外側で宣言することができます。このように宣言された変数は、**トップレベル**で宣言されたものと呼ばれます。

```
{style="tip"}
```

`customers` は可変変数であるため、宣言後にその値を再代入することができます。

デフォルトでは、すべての変数を読み取り専用 (`val`) として宣言することをお勧めします。変更可能な変数 (`var`) は、本当に必要な場合にのみ使用してください。そうすることで、変更するつもりではなかったものを誤って変更してしまう可能性が低くなります。

```
{style="note"}
```

文字列テンプレート

変数の内容を標準出力に表示する方法を知っておくと便利です。これは**文字列テンプレート**を使って行うことができます。テンプレート式を使用すると、変数やその他のオブジェクトに格納されているデータにアクセスし、それらを文字列に変換することができます。文字列値とは、二重引用符 `"` で囲まれた文字の列のことです。テンプレート式は常にドル記号 `$` で始まります。

テンプレート式内でコードを評価するには、ドル記号 `$` の後に中括弧 `{}` を置き、その中にコードを記述します。

例えば：

```
fun main() {
    //sampleStart
    val customers = 10
    println("There are $customers customers")
    // There are 10 customers

    println("There are ${customers + 1} customers")
    // There are 11 customers
    //sampleEnd
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-string-templates"}
```

詳細については、[文字列テンプレート](#)を参照してください。

変数に対して型が宣言されていないことにお気づきでしょう。Kotlin が自動的に型 (`Int`) を推論しています。このツアーでは、[次の章](#)で、Kotlin の基本型とその宣言方法について解説します。

練習 {completion-point= 「true」}

Exercise {initial-collapse-state="collapsed" collapsible="true"}

Cプログラムが標準出力に `"Mary is 20 years old"` を出力するように、コードを完成させてください：

|---|---|

```
fun main() {  
    val name = "Mary"  
    val age = 20  
    // Write your code here  
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-hello-world-exercise"}

|---|---|

```
fun main() {  
    val name = "Mary"  
    val age = 20  
    println("$name is $age years old")  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-hello-world-solution"}

- [Next step](#)